

CPU Scheduling Algorithms Lab Report

Name: Abu Bakar Siddique

Roll: 47

June 21, 2026

1 Theory

CPU scheduling decides which ready process will use the CPU next. In this lab, the following algorithms were implemented.

1.1 First-Come, First-Served

FCFS is a non-preemptive algorithm. Processes execute in the same order in which they arrive. For processes with arrival time 0:

$$WT_i = \sum_{j=1}^{i-1} BT_j$$

$$TAT_i = WT_i + BT_i$$

1.2 Shortest Job First

SJF is a non-preemptive algorithm. It selects the process with the smallest CPU burst time. If two processes have the same burst time, FCFS order is preserved.

1.3 Shortest Remaining Time Next

SRTN is the preemptive version of SJF. At each time unit, the process with the minimum remaining burst time is selected.

$$TAT_i = CT_i - AT_i$$

$$WT_i = TAT_i - BT_i$$

1.4 Round Robin

Round Robin is a preemptive scheduling algorithm. Each process gets the CPU for at most one time quantum. If it is not finished, it goes back to the ready queue.

1.5 Lottery Scheduling

Lottery Scheduling is probabilistic. Each process receives a number of tickets. At each scheduling decision, a random ticket is drawn, and the process holding that ticket gets the CPU for one time quantum.

2 Results

2.1 FCFS Result

Input burst times: 6, 8, 3, 7, 4 ms.

Table 1: FCFS scheduling output

Process	Burst Time	Waiting Time	Turnaround Time
P1	6	0	6
P2	8	6	14
P3	3	14	17
P4	7	17	24
P5	4	24	28

```
Enter number of processes: 5
Enter burst times:
P1: 6
P2: 8
P3: 3
P4: 7
P5: 4

===== FCFS Scheduling =====
Process    Burst(ms)    Waiting(ms)    Turnaround(ms)
-----
P1         6           0             6
P2         8           6            14
P3         3          14            17
P4         7          17            24
P5         4          24            28

Average Waiting Time    : 12.20 ms
Average Turnaround Time : 17.80 ms
```

Figure 1: FCFS output screenshot

Average waiting time = 12.20 ms. Average turnaround time = 17.80 ms.

2.2 SJF Result

The same process set was used for SJF.

Table 2: SJF scheduling output

Process	Burst Time	Waiting Time	Turnaround Time
P3	3	0	3
P5	4	3	7
P1	6	7	13
P4	7	13	20
P2	8	20	28

```
Enter number of processes: 5
Enter burst times:
P1: 6
P2: 8
P3: 3
P4: 7
P5: 4

===== SJF Scheduling =====
Process    Burst(ms)    Waiting(ms)    Turnaround(ms)
-----
P3         3           0             3
P5         4           3             7
P1         6           7            13
P4         7          13            20
P2         8          20            28

Average Waiting Time    : 8.60 ms
Average Turnaround Time : 14.20 ms
```

Figure 2: SJF output screenshot

Average waiting time = 8.60 ms. Average turnaround time = 14.20 ms.

2.3 FCFS and SJF Comparison

Table 3: Comparison between FCFS and SJF

Algorithm	Average Waiting Time	Average Turnaround Time
FCFS	12.20 ms	17.80 ms
SJF	8.60 ms	14.20 ms

For the same process set, SJF gives a lower average waiting time than FCFS because shorter jobs are executed earlier.

2.4 SRTN Result

Table 4: SRTN scheduling output

Process	Arrival	Burst	Completion	Waiting	Turnaround
P1	0	8	17	9	17
P2	1	4	5	0	4
P3	2	9	26	15	24
P4	3	5	10	2	7

```

Enter number of processes: 4
Enter arrival time and burst time for each process:
P1 arrival: 0
P1 burst: 8
P2 arrival: 1
P2 burst: 4
P3 arrival: 2
P3 burst: 9
P4 arrival: 3
P4 burst: 5

===== SRTN Scheduling =====
Process    Arrival    Burst      Completion    Waiting    Turnaround
-----
P1         0         8         17           9          17
P2         1         4          5           0           4
P3         2         9         26          15          24
P4         3         5         10           2           7

Average Waiting Time : 6.50 ms
Average Turnaround Time : 13.00 ms

```

Figure 3: SRTN output screenshot

Average waiting time = 6.50 ms. Average turnaround time = 13.00 ms. This demonstrates preemption because P1 starts first, but P2 arrives at time 1 with a shorter remaining time and preempts P1.

2.5 Round Robin Result

Input burst times: 24, 3, 3 ms.

Table 5: Round Robin comparison

Quantum	Average Waiting Time	Average Turnaround Time
4 ms	5.67 ms	15.67 ms
2 ms	6.33 ms	16.33 ms

```

PS E:\CSE6003-2\os\lab\Scheduling_Lab_report> .\scheduling.exe
Enter number of processes: 3
Enter burst times:
P1: 24
P2: 3
P3: 3
Enter time quantum: 4

===== Round Robin Scheduling (Quantum = 4 ms) =====
Gantt Chart: [P1: 0-4] [P2: 4-7] [P3: 7-10] [P1: 10-14] [P1: 14-18] [P1: 18-22] [P1: 22-26] [P1: 26-30]

Process  Burst(ms)  Waiting(ms)  Turnaround(ms)
-----
P1       24         6             30
P2        3         4             7
P3        3         7            10

Average Waiting Time : 5.67 ms
Average Turnaround Time : 15.67 ms

===== CPU Scheduling Menu =====
1. First-Come, First-Served (FCFS)
2. Shortest Job First (SJF)
3. Shortest Remaining Time Next (SRTN)
4. Round Robin (RR)
5. Lottery Scheduling
0. Exit
Enter your choice: 4

Enter number of processes: 3
Enter burst times:
P1: 24
P2: 3
P3: 3
Enter time quantum: 2

===== Round Robin Scheduling (Quantum = 2 ms) =====
Gantt Chart: [P1: 0-2] [P2: 2-4] [P3: 4-6] [P1: 6-8] [P2: 8-9] [P3: 9-10] [P1: 10-12] [P1: 12-14] [P1: 14-16] [P1: 16-18] [P1: 18-20] [P1: 20-22] [P1: 22-24] [P1: 24-26] [P1: 26-28] [P1: 28-30]

Process  Burst(ms)  Waiting(ms)  Turnaround(ms)
-----
P1       24         6             30
P2        3         6             9
P3        3         7            10

Average Waiting Time : 6.33 ms
Average Turnaround Time : 16.33 ms

```

Figure 4: Round Robin output screenshot

For this input, quantum 4 ms performs slightly better than quantum 2 ms in both average waiting time and average turnaround time.

2.6 Lottery Scheduling Result

Input: P1 burst = 10, tickets = 10; P2 burst = 6, tickets = 20; P3 burst = 4, tickets = 5; quantum = 2 ms.

Table 6: Lottery Scheduling three-run comparison

Run	Average Waiting Time	Average Turnaround Time
1	8.67 ms	15.33 ms
2	10.00 ms	16.67 ms
3	8.67 ms	15.33 ms

```

PS E:\CSED\3-2\OS\Lab\Scheduling_Lab_Report> .\scheduling.exe
===== Lottery Scheduling (Quantum = 2 ms) =====
Time  Winner  Ticket
-----
t=0   P2     ticket #25
t=2   P2     ticket #24
t=4   P2     ticket #20
t=6   P1     ticket #0
t=8   P3     ticket #10
t=10  P1     ticket #2
t=12  P1     ticket #7
t=14  P1     ticket #5
t=16  P3     ticket #11
t=18  P1     ticket #5

Process  Burst(ms)  Tickets  Completion  Waiting  Turnaround
-----
P1       10        10        20          10        20
P2       6         20         6           0         6
P3       4         5         18         14        18

Average Waiting Time : 8.00 ms
Average Turnaround Time : 14.67 ms

--- Lottery Run 2 ---
===== Lottery Scheduling (Quantum = 2 ms) =====
Time  Winner  Ticket
-----
t=0   P1     ticket #5
t=2   P2     ticket #21
t=4   P2     ticket #16
t=6   P3     ticket #11
t=8   P2     ticket #20
t=10  P3     ticket #14
t=12  P1     ticket #5
t=14  P1     ticket #0
t=16  P1     ticket #4
t=18  P1     ticket #2

Process  Burst(ms)  Tickets  Completion  Waiting  Turnaround
-----
P1       10        10        20          10        20
P2       6         20        10           4        10
P3       4         5         12           8        12

Average Waiting Time : 7.33 ms
Average Turnaround Time : 14.00 ms

--- Lottery Run 3 ---
===== Lottery Scheduling (Quantum = 2 ms) =====
Time  Winner  Ticket
-----
t=0   P3     ticket #30
t=2   P2     ticket #28
t=4   P2     ticket #28
t=6   P1     ticket #1
t=8   P2     ticket #10
t=10  P1     ticket #0
t=12  P3     ticket #13
t=14  P1     ticket #2
t=16  P1     ticket #5
t=18  P1     ticket #3

Process  Burst(ms)  Tickets  Completion  Waiting  Turnaround
-----
P1       10        10        20          10        20
P2       6         20        10           4        10
P3       4         5         14          10        14

Average Waiting Time : 8.00 ms
Average Turnaround Time : 14.67 ms

```

Figure 5: Lottery Scheduling output screenshot

The three runs produced different execution orders, which shows the probabilistic nature of Lottery Scheduling.

A Source Code

Listing 1: scheduling.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  #define MAX 10
6
7  void fcfs(int n, int burst[]) {
8      int wt[MAX], tat[MAX];
9      float avg_wt = 0, avg_tat = 0;
10
11     wt[0] = 0;
12     for (int i = 1; i < n; i++)
13         wt[i] = wt[i-1] + burst[i-1];
14
15     printf("\n===== FCFS Scheduling =====\n");
16     printf("%-10s %-15s %-15s %-15s\n", "Process", "Burst(ms)", "Waiting(ms)", "
17         Turnaround(ms)");
18     printf("-----\n");

```

```

19     for (int i = 0; i < n; i++) {
20         tat[i] = wt[i] + burst[i];
21         avg_wt += wt[i];
22         avg_tat += tat[i];
23         printf("P%-9d %-15d %-15d %-15d\n", i+1, burst[i], wt[i], tat[i]);
24     }
25
26     printf("\nAverage Waiting Time : %.2f ms\n", avg_wt / n);
27     printf("Average Turnaround Time : %.2f ms\n", avg_tat / n);
28 }
29
30
31 void sjf(int n, int burst[]) {
32     int b[MAX], idx[MAX], wt[MAX], tat[MAX];
33     float avg_wt = 0, avg_tat = 0;
34
35     for (int i = 0; i < n; i++) { b[i] = burst[i]; idx[i] = i+1; }
36
37     for (int i = 1; i < n; i++) {
38         int key_b = b[i], key_idx = idx[i], j = i - 1;
39         while (j >= 0 && b[j] > key_b) {
40             b[j + 1] = b[j];
41             idx[j + 1] = idx[j];
42             j--;
43         }
44         b[j + 1] = key_b;
45         idx[j + 1] = key_idx;
46     }
47
48     wt[0] = 0;
49     for (int i = 1; i < n; i++) wt[i] = wt[i-1] + b[i-1];
50
51     printf("\n===== SJF Scheduling =====\n");
52     printf("%-10s %-15s %-15s %-15s\n", "Process", "Burst(ms)", "Waiting(ms)", "
53         Turnaround(ms)");
54     printf("-----\n");
55
56     for (int i = 0; i < n; i++) {
57         tat[i] = wt[i] + b[i];
58         avg_wt += wt[i];
59         avg_tat += tat[i];
60         printf("P%-9d %-15d %-15d %-15d\n", idx[i], b[i], wt[i], tat[i]);
61     }
62
63     printf("\nAverage Waiting Time : %.2f ms\n", avg_wt / n);
64     printf("Average Turnaround Time : %.2f ms\n", avg_tat / n);
65 }
66
67 void srtn(int n, int arrival[], int burst[]) {
68     int rem[MAX], done[MAX], complete[MAX], wt[MAX], tat[MAX];
69     float avg_wt = 0, avg_tat = 0;
70
71     for (int i = 0; i < n; i++) { rem[i] = burst[i]; done[i] = 0; }
72
73     int finished = 0, t = 0;
74     while (finished < n) {
75         int sel = -1, minR = 99999;

```

```

76     for (int i = 0; i < n; i++)
77         if (!done[i] && arrival[i] <= t && rem[i] < minR) {
78             minR = rem[i]; sel = i;
79         }
80     if (sel == -1) { t++; continue; }
81     rem[sel]--;
82     t++;
83     if (rem[sel] == 0) {
84         done[sel] = 1;
85         complete[sel] = t;
86         finished++;
87     }
88 }
89
90 printf("\n===== SRTN Scheduling =====\n");
91 printf("%-10s %-12s %-12s %-14s %-12s %-14s\n",
92         "Process", "Arrival", "Burst", "Completion", "Waiting", "Turnaround");
93 printf("-----\n");
94 ;
95
96 for (int i = 0; i < n; i++) {
97     tat[i] = complete[i] - arrival[i];
98     wt[i] = tat[i] - burst[i];
99     avg_wt += wt[i];
100    avg_tat += tat[i];
101    printf("P%-9d %-12d %-12d %-14d %-12d %-14d\n",
102            i+1, arrival[i], burst[i], complete[i], wt[i], tat[i]);
103 }
104
105 printf("\nAverage Waiting Time : %.2f ms\n", avg_wt / n);
106 printf("Average Turnaround Time : %.2f ms\n", avg_tat / n);
107 }
108
109 void round_robin(int n, int burst[], int quantum) {
110     int rem[MAX], complete[MAX], wt[MAX], tat[MAX];
111     float avg_wt = 0, avg_tat = 0;
112
113     for (int i = 0; i < n; i++) rem[i] = burst[i];
114
115     int t = 0, finished = 0;
116     /* track finish time per process */
117     for (int i = 0; i < n; i++) complete[i] = 0;
118
119     printf("\n===== Round Robin Scheduling (Quantum = %d ms) =====\n", quantum);
120     printf("Gantt Chart: ");
121
122     while (finished < n) {
123         int did_work = 0;
124         for (int i = 0; i < n; i++) {
125             if (rem[i] > 0) {
126                 did_work = 1;
127                 int run = (rem[i] < quantum) ? rem[i] : quantum;
128                 printf("P%d: %d-%d ", i+1, t, t+run);
129                 t += run;
130                 rem[i] -= run;
131                 if (rem[i] == 0) { complete[i] = t; finished++; }
132             }

```



```

133     }
134     if (!did_work) break;
135 }
136
137 printf("\n\n%-10s %-15s %-15s %-15s\n", "Process", "Burst(ms)", "Waiting(ms)", "
    Turnaround(ms)");
138 printf("-----\n");
139
140 for (int i = 0; i < n; i++) {
141     tat[i] = complete[i];
142     wt[i] = tat[i] - burst[i];
143     avg_wt += wt[i];
144     avg_tat += tat[i];
145     printf("P%-9d %-15d %-15d %-15d\n", i+1, burst[i], wt[i], tat[i]);
146 }
147
148 printf("\nAverage Waiting Time : %.2f ms\n", avg_wt / n);
149 printf("Average Turnaround Time : %.2f ms\n", avg_tat / n);
150 }
151
152
153 void lottery(int n, int burst[], int tickets[], int quantum) {
154     int rem[MAX], complete[MAX];
155     float avg_wt = 0, avg_tat = 0;
156
157     for (int i = 0; i < n; i++) { rem[i] = burst[i]; complete[i] = 0; }
158
159     int t = 0, finished = 0;
160
161     printf("\n==== Lottery Scheduling (Quantum = %d ms) =====\n", quantum);
162     printf("%-8s %-10s %-10s\n", "Time", "Winner", "Ticket");
163     printf("-----\n");
164
165     while (finished < n) {
166         int total = 0;
167         for (int i = 0; i < n; i++) if (rem[i] > 0) total += tickets[i];
168         if (total == 0) break;
169
170         int draw = rand() % total;
171         int cum = 0, sel = -1;
172         for (int i = 0; i < n; i++) {
173             if (rem[i] <= 0) continue;
174             cum += tickets[i];
175             if (draw < cum) { sel = i; break; }
176         }
177
178         int run = (rem[sel] < quantum) ? rem[sel] : quantum;
179         printf("t=%-6d P%-6d ticket #%-6d\n", t, sel+1, draw);
180         t += run;
181         rem[sel] -= run;
182         if (rem[sel] == 0) { complete[sel] = t; finished++; }
183     }
184
185     printf("\n%-10s %-12s %-10s %-14s %-12s %-14s\n",
186         "Process", "Burst(ms)", "Tickets", "Completion", "Waiting", "Turnaround");
187     printf("
    -----\n")
    ;

```

```

188
189     for (int i = 0; i < n; i++) {
190         int tat = complete[i]; /* arrival = 0 */
191         int wt = tat - burst[i];
192         avg_wt += wt;
193         avg_tat += tat;
194         printf("P%-9d %-12d %-10d %-14d %-12d %-14d\n",
195             i+1, burst[i], tickets[i], complete[i], wt, tat);
196     }
197
198     printf("\nAverage Waiting Time : %.2f ms\n", avg_wt / n);
199     printf("Average Turnaround Time : %.2f ms\n", avg_tat / n);
200 }
201
202
203 int main(void) {
204     srand((unsigned)time(NULL));
205
206     int choice;
207
208     do {
209         printf("\n===== CPU Scheduling Menu =====\n");
210         printf("1. First-Come, First-Served (FCFS)\n");
211         printf("2. Shortest Job First (SJF)\n");
212         printf("3. Shortest Remaining Time Next (SRTN)\n");
213         printf("4. Round Robin (RR)\n");
214         printf("5. Lottery Scheduling\n");
215         printf("0. Exit\n");
216         printf("Enter your choice: ");
217         scanf("%d", &choice);
218
219         if (choice == 1) {
220             int n, burst[MAX];
221
222             printf("\nEnter number of processes: ");
223             scanf("%d", &n);
224
225             printf("Enter burst times:\n");
226             for (int i = 0; i < n; i++) {
227                 printf("P%d: ", i + 1);
228                 scanf("%d", &burst[i]);
229             }
230
231             fcfs(n, burst);
232         }
233         else if (choice == 2) {
234             int n, burst[MAX];
235
236             printf("\nEnter number of processes: ");
237             scanf("%d", &n);
238
239             printf("Enter burst times:\n");
240             for (int i = 0; i < n; i++) {
241                 printf("P%d: ", i + 1);
242                 scanf("%d", &burst[i]);
243             }
244
245             sjf(n, burst);

```

```

246     }
247     else if (choice == 3) {
248         int n, arrival[MAX], burst[MAX];
249
250         printf("\nEnter number of processes: ");
251         scanf("%d", &n);
252
253         printf("Enter arrival time and burst time for each process:\n");
254         for (int i = 0; i < n; i++) {
255             printf("P%d arrival: ", i + 1);
256             scanf("%d", &arrival[i]);
257             printf("P%d burst: ", i + 1);
258             scanf("%d", &burst[i]);
259         }
260
261         srtn(n, arrival, burst);
262     }
263     else if (choice == 4) {
264         int n, burst[MAX], quantum;
265
266         printf("\nEnter number of processes: ");
267         scanf("%d", &n);
268
269         printf("Enter burst times:\n");
270         for (int i = 0; i < n; i++) {
271             printf("P%d: ", i + 1);
272             scanf("%d", &burst[i]);
273         }
274
275         printf("Enter time quantum: ");
276         scanf("%d", &quantum);
277
278         round_robin(n, burst, quantum);
279     }
280     else if (choice == 5) {
281         int n, burst[MAX], tickets[MAX], quantum;
282
283         printf("\nEnter number of processes: ");
284         scanf("%d", &n);
285
286         printf("Enter burst time and tickets for each process:\n");
287         for (int i = 0; i < n; i++) {
288             printf("P%d burst: ", i + 1);
289             scanf("%d", &burst[i]);
290             printf("P%d tickets: ", i + 1);
291             scanf("%d", &tickets[i]);
292         }
293
294         printf("Enter time quantum: ");
295         scanf("%d", &quantum);
296
297         for (int run = 1; run <= 3; run++) {
298             printf("\n--- Lottery Run %d ---", run);
299             lottery(n, burst, tickets, quantum);
300         }
301     }
302     else if (choice == 0) {
303         printf("\nExiting program.\n");

```

```
304     }
305     else {
306         printf("\nInvalid choice. Please try again.\n");
307     }
308 } while (choice != 0);
309
310 return 0;
311 }
```